Machine Learning for Data Science:

Introduction to Machine Learning

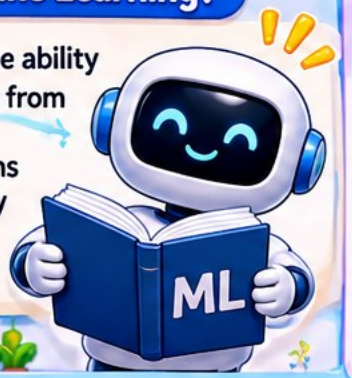
Let's explore how machines learn from data!

Machine Learning (ML) is a branch of Data Science that helps computers learn from data!

Instead of programming every rule, we let machines find patterns!

1. What is Machine Learning?

Machine Learning is the ability of a computer to learn from past data and make predictions or decisions without being explicitly programmed.



2. How It Works?

- **Data Collection** (Gather data)
- **Model Training** (Find patterns)
- **Prediction** (Make decisions)

3. Types of Machine Learning



Supervised Learning
(With labeled data)
- Classification
- Regression



Unsupervised Learning
(Without labels)
- Clustering
- Association



Reinforcement Learning
(Learn by rewards)
- Decision Making

4. Real-World Applications

- **Healthcare** (Disease Prediction)
- **E-commerce** (Product Recommendation)
- **Self-Driving Cars** (Object Detection)
- **Finance** (Fraud Detection)
- **Weather Forecasting** (Climate Analysis)

5. Popular ML Algorithms

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machines
- Neural Networks

These algorithms help us build smart models!

6. Example: Spam Email Detection

ML can learn to identify spam emails from features like keywords, sender, etc.



Normal (Inbox)



Spam (Junk)

7. Key Takeaways

- ML helps computers learn from data.
- It is widely used in real life.
- Common types: Supervised, Unsupervised, Reinforcement.
- Popular algorithms: Regression, Decision Trees, etc.
- ML is the heart of Data Science!

Start your ML journey today and turn data into insights!

Machine Learning = Smarter Decisions!

Types of Learning

Supervised, Unsupervised and Reinforcement Learning

Different ways for machines to learn!

Let's explore the 3 main types of Learning in Machine Learning!

Each type has a unique way to learn and solve problems!

1. Supervised Learning

Learn with Labeled Data

Uses input data with known labels to learn a mapping from inputs to outputs.

Examples



Spam Email Detection



Cat or Dog Classification



Stock Price Prediction

- **Input :** Features + Correct Labels
- **Goal :** Predict correct output
- **Algorithms :** Linear Regression, Logistic Regression, Decision Trees

2. Unsupervised Learning

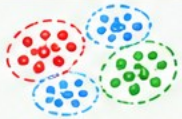
Find Patterns in Data

Uses unlabeled data to discover hidden patterns or groups.

Examples



Clustering (Customer Segmentation)



Dimensionality Reduction (PCA)



Association Rules (Market Basket)

- **Input :** Unlabeled Data
- **Goal :** Find structure or groups
- **Algorithms :** K-Means, PCA, Apriori

3. Reinforcement Learning

Learn by Rewards

Learns through trial and error by receiving rewards or penalties.

Examples



Game Playing (AlphaGo)



Robot Navigation



Self-Driving Cars

- **Inputt :** Actions and Rewards
- **Goal :** Maximize reward
- **Algorithms :** Q-Learning, Deep Q-Network (DQN)

Quick Comparison

Aspect	Supervised	Unsupervised	Reinforcement
Data Type	Labeled	Unlabeled	Reward-based
Goal	Predict	Discover Patterns	Maximize Reward
Examples	Classification	Clustering	Game/Robotics

Together, these learning types help machines classify, find patterns and make smart decisions!

Choose the Right Type for the Right Problem!

The Machine Learning Workflow

From Problem to Prediction!

Let's follow the ML workflow to build smart solutions!

This is the step-by-step journey of Machine Learning!

1. Define the Problem

Understand the goal and problem.



Example:
Predict house prices

2. Collect Data

Gather relevant data from sources.



Example:
House data from website

3. Prepare Data

Clean and transform the data.



- Handle missing values
- Encode categories
- Normalize features

4. Choose a Model

Select an appropriate ML algorithm.



- Linear Regression
- Decision Tree
- Random Forest
- Neural Network

5. Train the Model

Learn patterns from data.



- Fit the model
- Tune parameters

6. Evaluate Performance

Check how well it works.



- Accuracy
- Precision
- Recall
- RMSE

7. Deploy & Monitor

Use the model in real-world and track performance.



- Make predictions
- Monitor & retrain if needed

ML Workflow at a Glance



Key Points

- Good Data = Better Results
- Choose the Right Model
- Evaluate Carefully
- Keep Improving



In Short...

The ML Workflow helps us turn data into useful insights and intelligent decisions!

Data Preparation for ML – Feature Engineering

Turn Raw Data into Powerful Features!

Feature Engineering helps the model learn better!

Good features lead to better predictions!

1. What is Feature Engineering?

Feature Engineering is the process of creating, selecting and transforming features from raw data to improve the performance of ML models.

Raw Data

Age	Income	Gender
25	50K	M
32	60K	F
45	80K	M

Better Features

Age	Income	Purchased
25	50K	1
32	60K	0
45	80K	1

2. Why is it Important?

- Improves model accuracy
- Helps handle complex data
- Reduces training time
- Makes the model more robust

3. Common Techniques

- **Feature Scaling** (Standardize / Normalize)
- **Encoding Categorical Data** (A, B, C) (One-Hot / Label Encoding)
- **Feature Transformation** log(x) (Log, Box-Cox, etc.)
- **Feature Selection** (Filter / Wrapper)
- **Feature Creation** (New Features)
- **Handling Missing Values** (Mean / Mode / Imputation)

4. Popular Techniques in Detail

Scaling

$$x' = \frac{(x - \mu)}{\sigma}$$

(Standardization)

Encoding

(One-Hot Encoding)

Feature Creation

Age^2, Income/Age

(Derived Features)

Dimensionality Reduction

(PCA)

5. Steps in Feature Engineering

1. Understand the Data (Explore & Analyze)
2. Handle Missing Values
3. Transform Features (Scale, Encode, etc.)
4. Create New Features
5. Select Important Features

Better Features = Better Predictions!

6. Real-World Applications

- **Healthcare** (Disease Prediction)
- **E-commerce** (Customer Behavior)
- **Self-Driving Cars** (Road Features)

Feature Engineering is a key step in the Machine Learning journey!

Feature Scaling & Train-Test Split

Prepare Data the Right Way for Better Machine Learning!

Before training your model, let's scale the features and split the data!

Clean + Scale + Split = Better Model!

1. What is Feature Scaling?

Feature Scaling is the process of bringing all features to a similar scale or range.

Why?

- Prevents large values from dominating
- Helps faster convergence of algorithms
- Improves model performance



Common Methods:

Standardization	Normalization	Min-Max Scaling
$x' = \frac{x - \mu}{\sigma}$ (Mean = 0, Std = 1)	$x' = \frac{x - x_{min}}{x_{naax} - x_{min}}$ (Range: 0 – 1)	0 1

2. Train-Test Split

Train-Test Split divides your data into two parts:

Training Set (Usually 70–80%)	Test Set (Usually 20–30%)
Used to train the model	Used to evaluate performance

Why Split?

- Check how well the model generalizes on new data
- Avoid overfitting
- Get unbiased performance estimate

3. How to Do it in Python?

```
from sklearn.model_selection
import train_test_split
from sklearn.preprocessing
import StandardScaler
```

1. **Feature Scaling:**

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

2. **Train-Test Split:**

```
X_train, X_test, y_train, y_test =
 train_test_split(X_scaled, y,
 test_size=0.2, random_state=42)
```

4. Key Takeaways

- Feature Scaling keeps features on the same scale.
- Train-Test Split helps evaluate your model fairly.
- Use StandardScaler, MinMaxScaler, or Normalization.
- Common split ratio: 70-80% train, 20-30% test.

5. Quick Comparison

Method	Range / Effect	Best For
Standardization	Mean 0, Std 1	Many ML models
Min-Max Scaling	0 to 1	Neural Networks
Normalization	Unit norm	Distance-based methods

6. Final Tip!

Scale your features and split your data before training! This simple step leads to a more accurate and reliable ML model.

Well Prepared Data = Smarter Predictions!

Happy Learning!

Cross-Validation

Get a Reliable Estimate of Model Performance!

What is Cross-Validation?

Cross-Validation is a technique to evaluate a machine learning model more reliably.

It helps us check how well our model generalizes on unseen data!

Smarter Evaluation!

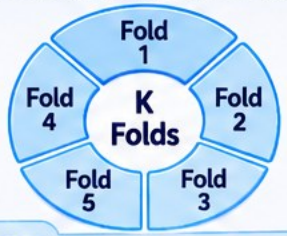
Why Do We Need It?

- Avoid overfitting
- Use all data for training & testing
- Get a more accurate performance estimate

Common Types of Cross-Validation

1. K-Fold Cross-Validation

Split data into K equal parts, use each part once for testing.



Steps:

1. Split data into K folds.
2. Train on K-1 folds.
3. Test on the remaining fold.

2. Stratified K-Fold

Keeps the same class ratio in each fold.



Best for:

- Classification (proportional classes).

3. Leave-One-Out Cross-Validation (LOOCV)

Use one sample for testing and the rest for training.



Good for:

- Small datasets.
- Maximizing training data.

4. Time Series Cross-Validation

Use past data to predict future data (no shuffling).



Ideal for:

- Time-dependent data (stock prices, weather).

Key Benefits

- More reliable performance estimate
- Reduces overfitting
- Works well with limited data



Quick Comparison

Method	Idea	Best For
K-Fold	Split into K parts	General use
Stratified K-Fold	Preserve class ratio	Classification
LOOCV	Use one sample	Small datasets
Time Series CV	Use past data	Time series

Things to Remember

- Choose the right method for your data.
- Use Stratified K-Fold for imbalanced classes.
- For time series, keep the time order.
- After CV, use the average score for final result.

Cross-Validation = Better Evaluation = More Trust in Your Model!

Supervised Learning – Linear Regression

Predict Continuous Values with a Best-Fit Line!

Linear Regression is a simple yet powerful Supervised Learning algorithm!

It helps us predict real numbers like price, salary, or temperature!

What is Linear Regression?

Linear Regression is a Supervised Learning technique used to predict a continuous output variable by finding a linear relationship between input features and output.

Example:

Predict house price based on area (sq.ft).



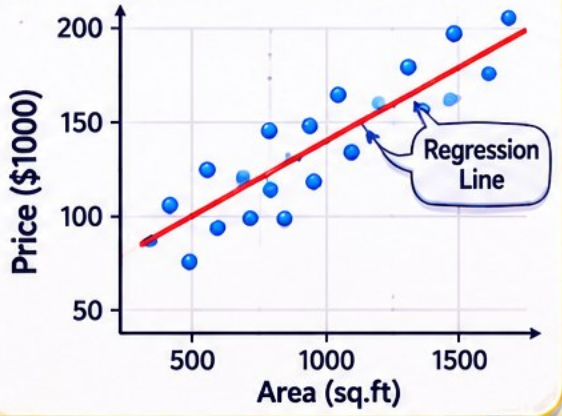
The Model

$$y = \beta_0 + \beta_1 x$$

Where:

- y – predicted value
- x – input feature
- β_0 – intercept
- β_1 – slope (coefficient)

Best-Fit Line



How It Works? (Steps)

1. **Prepare Data** (Handle missing values, scale if needed).
2. **Train the Model** (Learn the best-fit line).
3. **Make Predictions** (Estimate new values).
4. **Evaluate Performance** (Use metrics like MSE, R^2).

Key Concepts

Loss Function (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y - \hat{y})^2$$

Measures error

Gradient Descent



Finds the best parameters

R^2 Score



Shows how well the model fits

Overfitting & Underfitting



Balance model complexity

Advantages

- Simple and easy to understand
- Fast training
- Good for prediction tasks

Limitations

- Works only for linear relationships
- Sensitive to outliers

Real-World Applications



House Price Prediction



Salary Prediction



Temperature Forecasting



Stock Price Prediction

Key Takeaway:

Linear Regression helps us predict continuous values using a straight-line model!

Supervised Learning – Logistic Regression

Predict Probabilities, Classify with Confidence!

Logistic Regression is used for classification problems where the output is categorical (usually 0 or 1)!

It predicts the probability of a class using the sigmoid function!

Probability → Class (0 or 1)

1. What is Logistic Regression?

Logistic Regression is a supervised learning algorithm used for binary classification (e.g., pass/fail, spam/not spam, disease/no disease).

Key Idea:

- It models the probability of the positive class.
- Uses the sigmoid function to map values between 0 and 1.
- Predicts class based on a threshold (probability ≥ 0.5).

Example Applications

- Spam Detection
- Disease Prediction
- Credit Approval
- Student Pass/Fail

2. The Logistic Regression Model

Linear Equation:

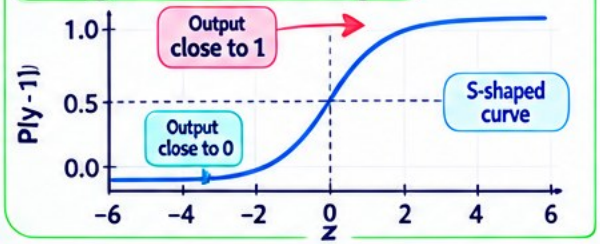
$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n = w^T x$$

Sigmoid Function:

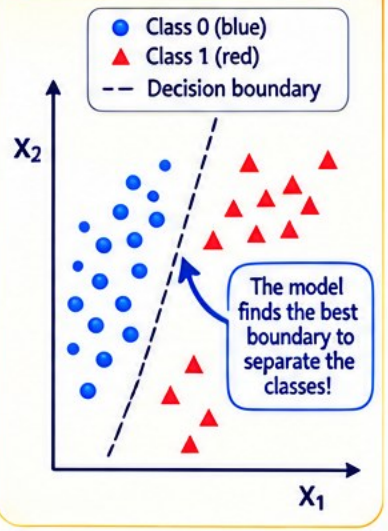
$$h(x) = \frac{1}{1 + e^{-z}}$$

Where,
- x = input features
- β = model parameters (weights)
- z = linear combination of inputs
- $h(x)$ = predicted probability (0 to 1)

3. Sigmoid Function Graph



4. Decision Boundary



5. Python Code Example

```python
# Import libraries
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create and train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, pred))
```

6. Steps to Implement

1. **Prepare Data**
   - Handle missing values
   - Scale features (optional)
2. **Split Data**
   - Train set (e.g., 80%)
   - Test set (e.g., 20%)
3. **Train the Model**
   - Fit the model using training data
   - Learn parameters (β values)
4. **Make Predictions**
   - Get probabilities using sigmoid function
5. **Evaluate Performance**
   - Accuracy
   - Precision/Recall
   - Confusion Matrix

7. Key Concepts

- Binary classification (0 or 1)
- Uses linear regression + sigmoid
- Outputs probability (0 to 1)
- Decision based on threshold (usually 0.5)
- Loss Function: Log Loss (Binary Cross-Entropy)

10. Real-World Applications

- Email Spam Detection
- Medical Diagnosis
- Credit Risk Assessment
- Customer Churn Prediction

7. Key Concepts

- Binary classification
- Uses linear regression + sigmoid
- Outputs probability (0 to 1)
- Decision based on threshold (usually 0.5)

8. Advantages

- Simple to implement
- Works well with high-dimensional data
- Provides probability estimates
- Interpretable and easy to explain

9. Limitations

- Assumes linear relationship between features and log odds
- Not suitable for complex non-linear data
- Sensitive to outliers

9. Limitations

Logistic Regression turns numbers into real-world decisions!

k-Nearest Neighbours

Classify by Your Nearest Neighbours!

k-NN is a simple yet powerful algorithm for classification and regression!

It works by finding the "k" closest data points!

Simple & Effective!

1. What is k-Nearest Neighbours?

k-Nearest Neighbours (k-NN) is a non-parametric, lazy learning algorithm that classifies a new data point based on the majority label of its k nearest neighbours.

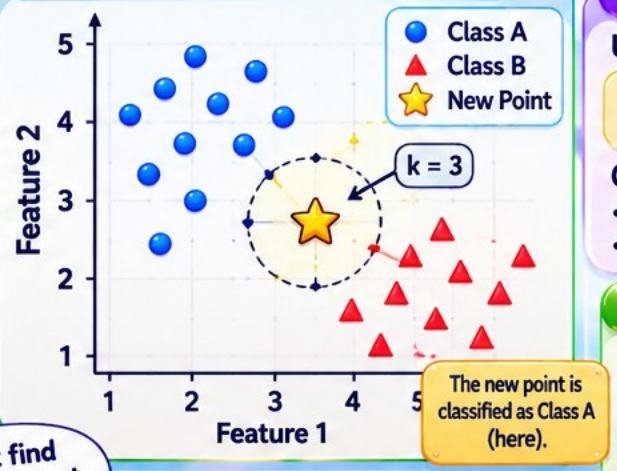
Key Ideas:

- Find the k closest points
- Use majority vote (classification) or average (regression)

2. How It Works?

1. Choose a value for k.
2. Calculate distance between the new point and all training points (e.g., Euclidean distance).
3. Select the k closest points.
4. Predict the class (majority vote) or value (average).

3. Example: Classification



The new point is classified as Class A (here).

4. Distance Measure

Usually Euclidean Distance:

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Other options:

- Manhattan Distance
- Minkowski Distance

5. Choosing k

- Small k → Sensitive to noise (may overfit).
- Large k → Smoother result (may underfit).
- Common choice: k = 3, 5, 7...

6. Types of k-NN

Classification

Predict a category (e.g., Spam or Not, Disease or Not).

Regression

Predict a continuous value (e.g., House Price).

7. Advantages

- Simple and easy to understand
- No training phase
- Works for both classification & regression
- Flexible with data

Just find your nearest friends!

8. Limitations

- Slow for large datasets
- Sensitive to irrelevant features
- Performance drops with high dimensions

9. Real-World Applications

- Medical Diagnosis
- Recommendation Systems
- Handwriting Recognition
- Location Prediction

k-NN – A Neighbourhood Approach to Smart Predictions!

Decision Trees

A Tree-Based Approach for Making Decisions!

What is a Decision Tree?

A Decision Tree is a flowchart-like model that helps us predict an output or make a decision.

It splits data based on features until we get a final decision!

Key Concepts

- **Root Node :** The topmost node (with the best feature).
- **Decision Node :** A node that tests a feature.
- **Branch :** The outcome of a test (e.g., Yes / No).
- **Leaf Node :** Final decision (e.g., Class label).

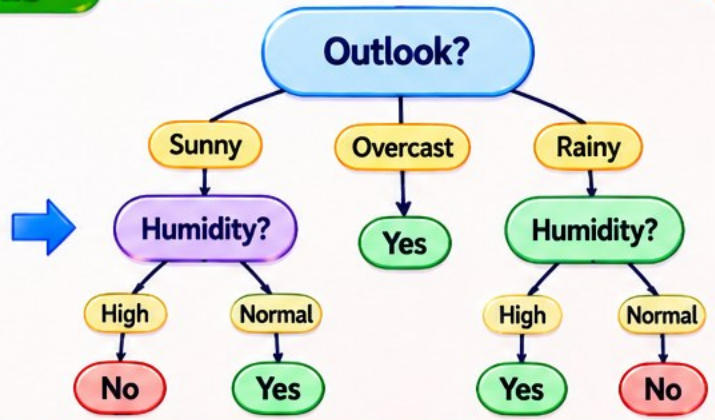
How It Works?

1. Choose the best feature (using criteria like Information Gain or Gini Index).
2. Split the data into branches.
3. Repeat the process for each branch until reaching a leaf node.
4. Predict the output from the leaf node.

Example: Play Tennis

Dataset (Simplified)

Outlook	Humidity	Play?
Sunny	High	No
Sunny	High	No
Overcast	Low	Yes
Rainy	High	Yes
Rainy	Normal	Yes
Overcast	High	Yes



Splitting Criteria

Information Gain

Choose the feature that reduces uncertainty the most.

$$IG = H(S) - \sum_{v} \frac{|S_v|}{|S|} H(S_v)$$

Gini Index

Choose the feature with lowest impurity.

$$\text{Gini} = 1 - \sum_{i} p_i^2$$

Types of Decision Trees

- **ID3 :** Uses Information Gain.
- **C4.5 :** Improves ID3 with pruning.
- **CART :** Uses Gini Index and supports both classification & regression.

Advantages

- Easy to understand and interpret.
- Works for both classification & regression.
- Handles categorical and numerical data.
- Requires little preprocessing.

Limitations

- Can overfit (with deep trees).
- Less accurate than some advanced models (e.g., SVM, Neural Nets).
- Sensitive to small data changes.

Real-World Applications

- Fraud Detection
- Customer Segmentation
- Healthcare Diagnosis
- Credit Risk
- Spam Detection
- Loan Approval

In Short: Decision Trees split data based on features to make simple and effective decisions!

Unsupervised Learning – k-Means Clustering

Find Groups in Data Without Labels!

k-Means Clustering is a simple method to group similar data points!

It divides data into k clusters based on similarity!

1. What is k-Means?

k-Means is an unsupervised learning algorithm that divides data into **k clusters**, where each cluster has similar data points.

Key Idea:

Minimize the distance between data points and their cluster centroid (mean).

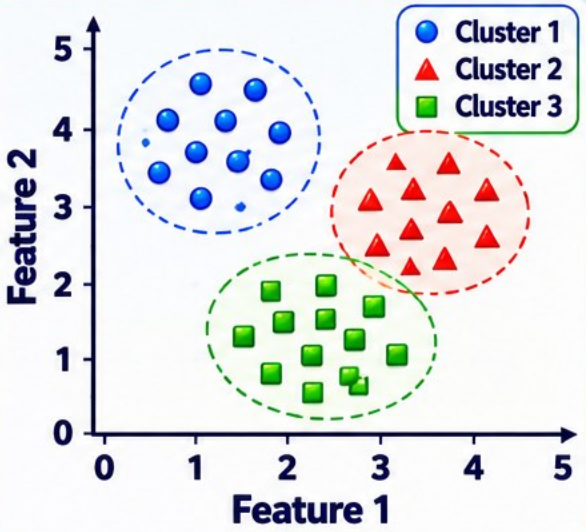
2. How It Works?

- Choose k (number of clusters).
- Initialize k centroids randomly.
- Assign each point to the nearest centroid.
- Update centroids as the mean of assigned points.
- Repeat until convergence.

3. Steps in Detail

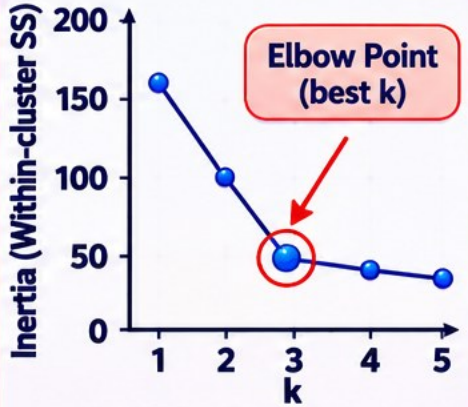
1. **Choose k:** Decide how many clusters you want.
2. **Initialize Centroids:** Pick k points randomly.
3. **Assign Points:** Each data point goes to the nearest centroid (using Euclidean distance).
4. **Update Centroids:** Recalculate the mean of each cluster.
5. **Repeat:** Steps 3 & 4 until centroids stop changing.

4. Example: 2D Clustering



5. Choosing k

Use the Elbow Method:



Pick the k at the elbow point (where the decrease slows down).

6. Key Concepts

Centroid
Mean point of a cluster.

Distance Metric
Usually Euclidean distance.

Clusters
Groups of similar data points.

7. Advantages

- Simple and easy to implement.
- Works well for large datasets.
- Useful for exploratory data analysis.

8. Limitations

- Need to specify k.
- Sensitive to initial centroids.
- Works best with spherical clusters.

9. Real-World Applications

- Customer Segmentation
- Image Compression
- Social Media Grouping
- Document Clustering

k-Means Clustering – A powerful and simple technique to find hidden patterns in data!

Hierarchical Clustering

Find Clusters in a Hierarchy!

Hierarchical Clustering builds a tree-like structure of clusters (dendrogram)!

It shows how clusters are merged or split step by step!

1. What is Hierarchical Clustering?

A method that creates a hierarchy of clusters, either by:

- **Agglomerative (Bottom-Up)** – merge smaller clusters
- **Divisive (Top-Down)** – split larger clusters

2. Key Concepts

- **Dendrogram** – Tree diagram of clusters.
- **Linkage** – Measure to combine clusters (e.g., single, complete, average).
- **Distance Metric** – e.g. Euclidean distance.

3. How It Works?

1. Start with each data point as its own cluster.
2. Merge the closest clusters (or split).
3. Repeat until one cluster (or desired number) remains.

4. Example: Dendrogram



5. Types / Methods

Agglomerative (Bottom-Up)

- Start with single points, merge closest clusters.
- Common linkage: Single, Complete, Average, Ward.

Divisive (Top-Down)

- Start with one cluster, split into smaller ones.
- Uses criteria like max variance.

6. Advantages

- No need to predefine number of clusters.
- Produces a hierarchy (dendrogram).
- Works with different distance measures.

7. Limitations

- Computationally expensive (for large datasets).
- Sensitive to noise and outliers.
- Not suitable for very large datasets.

8. Applications

- Market Segmentation
- Bioinformatics
- Document Clustering
- Medical Data Analysis

In Short:

Hierarchical Clustering builds a tree of clusters using merge or split steps, shown by a **dendrogram!**

Dimensionality Reduction with PCA

Reduce Dimensions, Keep the Essence!

PCA helps us compress data while retaining important information!

Fewer Features, Same Insights!

1. What is PCA?

PCA (Principal Component Analysis) is a technique to reduce the number of features by finding new uncorrelated variables (principal components) that capture the maximum variance in the data.

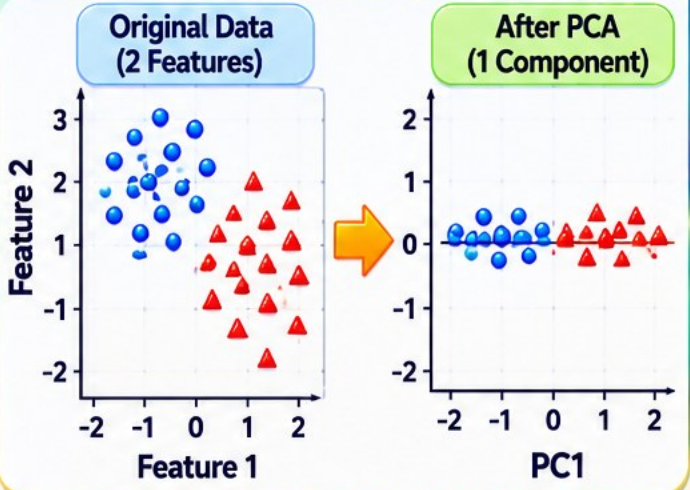
2. Key Concepts

- **Principal Components (PCs):** New features.
- **Variance:** Amount of information.
- **Orthogonal:** Uncorrelated PCs.
- **Explained Variance Ratio:** How much variance is captured.

3. Steps in PCA

1. Standardize the data (mean = 0, variance = 1).
2. Compute the covariance matrix.
3. Find eigenvalues & eigenvectors.
4. Choose top k components (with highest eigenvalues).
5. Project the data onto new subspace.

4. Before and After



5. How It Works?

PCA uses Singular Value Decomposition (SVD) to find directions (principal components) that maximize the variance of the data.

$$X = U\Sigma V^T$$

Project data onto top k PCs and get a lower-dimensional representation.

6. Advantages

- Reduces dimensionality.
- Retains most information.
- Helps with visualization.
- Improves model performance.

7. Limitations

- May lose some important information.
- Not always easy to interpret.
- Assumes linear relationships.

8. Applications

- Data Visualization
- Image Compression
- Face Recognition
- Medical Data Analysis

In Short:

PCA is a powerful technique to reduce dimensions, simplify data and keep the most important patterns!

Model Evaluation

Overfitting and Underfitting

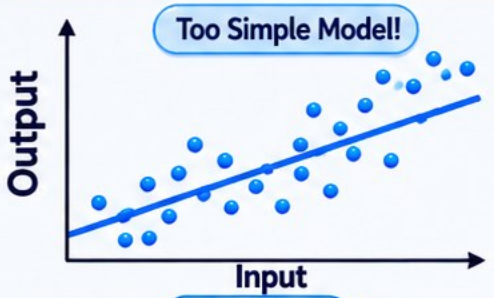
Find the Right Balance!

Good model should generalize well, not just memorize!

Avoid Overfitting and Underfitting!

Underfitting

Model is too simple to capture the true pattern.

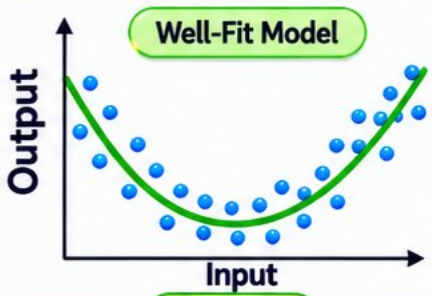


Signs:

- High Bias
- Poor Performance (on training & test data)

Good Fit

Model captures the right pattern.

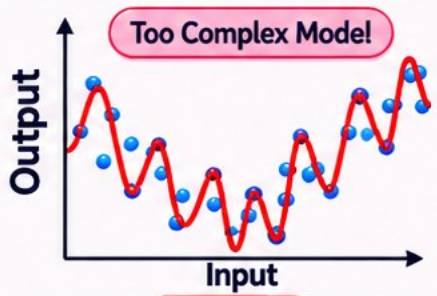


Signs:

- Low Bias, Low Variance
- Good Performance (on train & test data)

Overfitting

Model is too complex and learns noise.



Signs:

- Low Bias, High Variance
- Excellent Train Performance, Poor Test Performance

Key Takeaways:

- Underfitting: Too Simple → Increase Model Complexity
- Overfitting: Too Complex → Use Regularization, More Data, or Simpler Model
- Goal: Balance for Better Generalization!

Evaluation Metrics:

- Accuracy
- Precision & Recall
- F1-Score
- R^2 (for Regression)

How to Fix?

- For Underfitting: Use more features, complex models (e.g., higher degree, more layers).
- For Overfitting: Apply regularization (L1/L2), get more data, or use cross-validation.

Remember:

The Best Model is Neither Too Simple Nor Too Complex!

Evaluate Well... Build a Model that Learns and Generalizes!

Confusion Matrix, Accuracy, Precision

Evaluate Your Model Better!

Confusion Matrix helps us see how well our model makes predictions!

And Accuracy, Precision tell us the performance in numbers!

1. What is a Confusion Matrix?

A table that shows the correct and incorrect predictions.

Example:

		Predicted	
		Positive	Negative
Actual	Positive	TP (50)	FN (10)
	Negative	FP (5)	TN (35)

2. Key Metrics

Accuracy

Overall correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Example:

$$\text{Accuracy} = \frac{50 + 35}{50 + 35 + 5 + 10} = 85\%$$

Precision

How many predicted positives are correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Example:

$$\text{Precision} = \frac{50}{50 + 5} = 90.9\%$$

Recall (Sensitivity)

How many actual positives are found?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Example:

$$\text{Recall} = \frac{50}{50 + 10} = 83.3\%$$

3. Why It Matters?

- Helps compare models.
- Finds errors (FP & FN).
- Choose the right model for your problem.

4. Quick Summary

- Confusion Matrix → See TP, TN, FP, FN
- Accuracy → Overall performance
- Precision → Correct positive predictions
- Recall → Find all actual positives

Use these metrics to build a more reliable model!

Better Metrics = Better Decisions!

Key Takeaway:

Confusion Matrix + Accuracy + Precision = A Clear Picture of Your Model!



Recall

Recall measures how well our model finds all the actual positive cases.

$$\text{Recall} = \frac{TP}{TP + FN}$$

High Recall means fewer missed positives!



Example:

Recall = 0.90
→ 90% of actual positives are correctly found.

F1-Score

F1-Score is the harmonic mean of Precision and Recall.

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

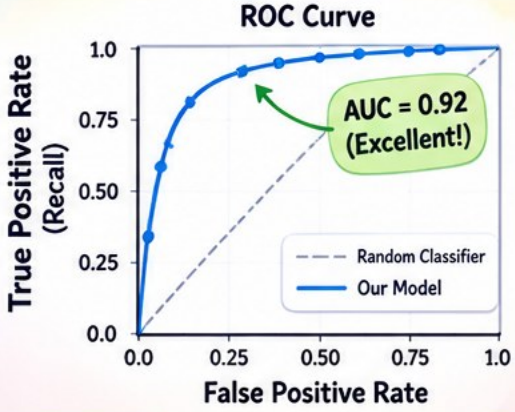
It balances false positives and false negatives.

When to use?

- ✓ When data is imbalanced.
- ✓ When both Precision & Recall matter!

ROC Curve

ROC Curve shows the trade-off between True Positive Rate (Recall) and False Positive Rate.



Key Points:

- ⭐ AUC = 1 → Perfect Model
- ⭐ AUC = 0.5 → Random Guess
- ⭐ Higher AUC is Better!

In short...

- ✓ **Recall:** Find all positives
- ✓ **F1-Score:** Balance Precision & Recall
- ✓ **ROC Curve:** Check performance across thresholds

Choose the right metric for the right problem!

Introduction to scikit-learn – Building



1. Import

```
from sklearn import datasets, model_selection, linear_model
```

Start by importing the necessary modules.

2. Load Data

```
iris = datasets.load_iris()
X, y = iris.data, iris.target
```

Use built-in datasets like Iris, Wine, etc.

3. Build a Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Train your model with ease!

4. Make Predictions

```
pred = model.predict(X_test)
```

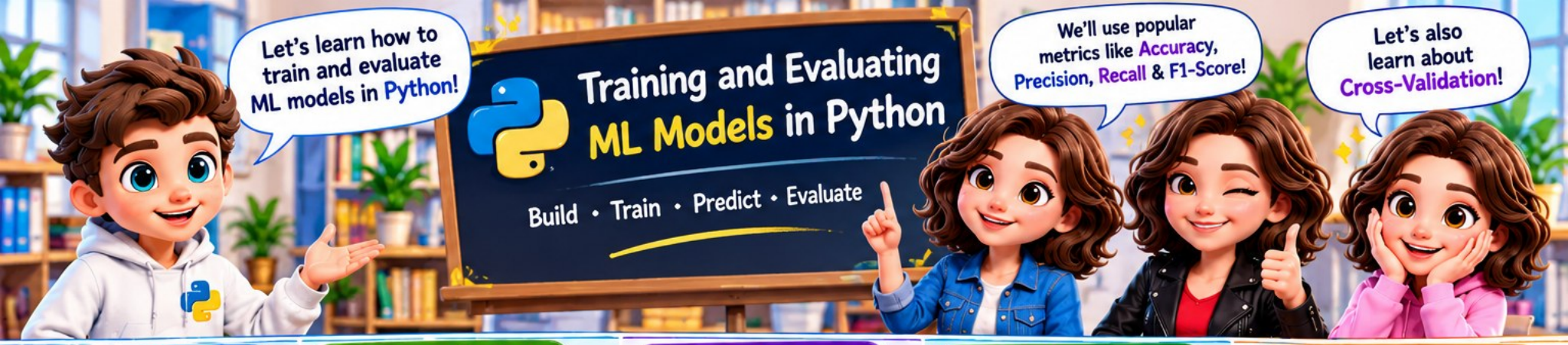
Get predictions on new data.

5. Evaluate

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, pred)
```

Check performance using metrics like accuracy, precision, recall, etc.

scikit-learn makes machine learning simple, scalable and beginner-friendly!



1. Import Libraries

Start by importing necessary libraries.

```
import pandas as pd
from sklearn.model_selection
    import train_test_split
from sklearn.linear_model
  import LogisticRegression
from sklearn.metrics import
accuracy_score, precision_score,
recall_score, f1_score
```

pandas | scikit learn

2. Prepare the Data

- Load dataset & handle missing values.
- Split into train and test sets. (e.g. 80% train, 20% test)

```
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
 random_state=42)
```

80% Train
20% Test

3. Train the Model

Choose a suitable algorithm and train your model.

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Training teaches the model patterns from data!

4. Make Predictions

Use the trained model to predict on new data.

```
y_pred = model.predict(X_test)
```

Actual	Predicted
0	0
1	1
0	0
1	1

Now we can compare the predictions!

5. Evaluate the Model

Use metrics to measure performance.

- **Accuracy: 95%** (Overall correctness)
- **Precision: 93%** (Few false positives)
- **Recall: 92%** (Finds most positives)
- **F1-Score: 92.5%** (Balance of Precision & Recall)

6. Understand Key Concepts

Training Set
Used to train the model.

Test Set
Used to evaluate performance.

Overfitting
Model too perfect on training data.

Underfitting
Model too simple to learn patterns.



7. Cross-Validation

Get a reliable estimate using K-Fold Cross-Validation.

```
from sklearn.model_selection
import cross_val_score

scores = cross_val_score(model, X, y,
cv=5, scoring='accuracy')
print("CV Accuracy:", scores.mean())
```

Reduce variance and get robust results!

8. Tune Hyperparameters

Improve performance with GridSearchCV or RandomizedSearchCV.

```
from sklearn.model_selection import
GridSearchCV

param_grid = {'C': [0.1, 1, 10],
      'solver': ['liblinear']}
grid = GridSearchCV(LogisticRegression(),
param_grid, cv=5)
grid.fit(X_train, y_train)
```

Find the best parameters!

9. Key Takeaways

- Use train/test split or cross-validation.
- Evaluate with multiple metrics (Accuracy, Precision, Recall, F1-Score).
- Avoid overfitting & underfitting.
- Tune hyperparameters for better results.
- Scikit-learn makes ML easy!

Train smart. Evaluate well. Build better ML models!